OpenZeppelin | security

UltraHonk Verifier Audit



September 18, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 6

Security Model and Trust Assumptions 7

Privileged Roles 8

Low Severity 9

L-01 Ignored Limb Padding Makes Proof Bytes Malleable for G1 Coordinates 9

L-02 Non-Canonical Scalar Encodings Make Proof Bytes Malleable 9

L-03 Conflicting Barretenberg Version Claims Leave the Supported Toolchain Ambiguous 10

L-04 Unvalidated BN254 G1 Points Either Trap Host Crypto or Escape Validation Entirely 11

L-05 Proofs Are Documented as ZK While the Verifier Implements Only the Non-ZK Flavor 11

Notes & Additional Information 12

N-01 Misleading Comments 12

N-02 Identity Example Does Not Provide Caller Binding or Replay Protection 13

N-03 The Verifier API Carries Two Env Handles and Mixes Them Within One Call 14

N-04 lookup_read_tags Is Not Constrained to Be Boolean 15

N-05 Unused Proof Padding Affects Fiat-Shamir Challenges Without Being Constrained 15

N-06 Circuit Build Inputs Are Not Immutably Pinned, So Artifact Reproducibility Is Not Guaranteed 16

Conclusion 18

Appendix 19

Issue Classification 19

Summary

Type	Cryptography
Timeline	From 2026-08-10 To 2026-08-28
Languages	Rust

Total Issues	11 (11 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	5 (5 resolved)
Notes & Additional Information	6 (6 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited the NethermindEth/rs-soroban-ultrahonk repository at commit `661db07`.

In scope were the following files:

```
crates
└── ultrahonk-soroban-verifier
    └── src
        ├── ec.rs
        ├── field.rs
        ├── hash.rs
        ├── lib.rs
        ├── relations.rs
        ├── shplemini.rs
        ├── sumcheck.rs
        ├── transcript.rs
        ├── types.rs
        ├── utils.rs
        └── verifier.rs
contracts
├── identity
│   └── src
│       └── lib.rs
└── rs-soroban-ultrahonk
    └── src
        └── lib.rs
circuits
├── fib_chain
│   └── src
│       └── main.nr
├── identity
│   └── src
│       └── main.nr
├── lookup_heavy
│   └── src
│       └── main.nr
├── many_pubs
│   └── src
│       └── main.nr
├── range_heavy
│   └── src
│       └── main.nr
├── simple_circuit
│   └── src
│       └── main.nr
├── small_circuit
│   └── src
```

```
|       └── main.nr
└── tornado
    └── src
        └── main.nr
```

All other files were out of scope, including `crates/ultrahonk-soroban-verifier/src/debug.rs`, the `contracts/tornado_classic` mixer contract, test utilities, tests, and scripts.

***Update**: The fixes for the findings highlighted in this report are all present at commit 15703c1, on the `audit` branch of the audited repository (pull request #41).*

System Overview

The repository provides an on-chain verifier for UltraHonk proofs on the Stellar network. UltraHonk is the proving system of Barretenberg, the backend of the Noir circuit language. The verifier crate is a Rust `no_std` port of the Barretenberg v0.87.0 UltraHonk verifier, specifically its non-zero-knowledge, Keccak-transcript flavor, targeting Soroban's Wasm environment. It allows Soroban contracts to check proofs produced off-chain for Noir circuits.

Verifier crate: The `ultrahonk-soroban-verifier` crate exposes a pure verification function that is deterministic over a verification key, a proof, and the public inputs, with no persistent state. Both the verification key (1760 bytes) and the proof (456 32-byte words) have fixed byte layouts that are length-checked and parsed before use, with additional consistency checks on verification-key metadata. Verification follows the upstream protocol: a Keccak-256 Fiat-Shamir transcript absorbs the public inputs and, round by round, each prover message before the challenges that depend on it are drawn, a sumcheck pass evaluates the full Ultra relation set (arithmetic, permutation, lookup, delta-range, elliptic, auxiliary, and Poseidon2 subrelations), and the Shplemini batch-opening scheme (Shplonk combined with Gemini) reduces all opening claims to a single KZG pairing check. The heavy cryptographic operations, BN254 field arithmetic, multi-scalar multiplication, the pairing check, and Keccak-256 hashing, are delegated to the native Soroban host functions; the crate itself implements only the surrounding logic such as byte conversion, field inversion, and protocol constants.

Contract wrappers: Two thin Soroban contracts embed the crate. `UltraHonkVerifierContract` exposes a generic `verify_proof` entry point, while `IdentityContract` exposes `prove_identity` for the identity circuit. Each contract stores a single immutable verification key in instance storage at deployment time via its constructor, after fixed-layout parsing and consistency checks. A `vk_bytes` getter exposes the stored key for external inspection. The verification entry points are permissionless and stateless.

Circuits: Eight Noir circuits accompany the verifier. Six of them (`fib_chain`, `lookup_heavy`, `many_pubs`, `range_heavy`, `simple_circuit`, and `small_circuit`) are small fixtures that exercise different relation families and proof shapes. The `identity` circuit proves that a private preimage hashes to a public Poseidon2 value. The `tornado` circuit implements a Tornado-style mixer statement, proving Merkle-tree membership of a note

commitment and correct derivation of its nullifier; its consuming mixer contract lives in the out-of-scope `tornado_classic` directory.

Security Model and Trust Assumptions

Verification is permissionless: any caller fully controls the proof and public-input bytes it submits, so the soundness of the port is the system's entire defense. The audit accordingly centered on a differential comparison against the Barretenberg v0.87.0 verifier, covering transcript and challenge derivation, the relation algebra, the batch-opening scheme, and the byte-level input handling.

Deployer: The constructor is the only privileged action and can be performed once. The deployer is trusted to supply the correct verification key for the intended circuit: the contract checks its fixed layout and selected metadata, but not its correspondence to any particular circuit. Anyone can deploy an instance with an arbitrary key, so callers must independently verify the stored key, exposed through `vk_bytes`, against the expected key for the intended circuit and must not rely on the contract address alone as a trust anchor. After deployment, the deployer holds no power.

Soroban host: The host-native BN254 operations and Keccak-256 hash are trusted to be correct and available. The verifier delegates group operations, including multi-scalar multiplication and the final pairing check, to the Soroban host rather than implementing them in the crate. The security of the Fiat-Shamir transcript further rests on the collision and preimage resistance of Keccak-256.

Prover toolchain: Proofs and verification keys must be produced by a toolchain compatible with Barretenberg 0.87.0 using the `ultra_honk` scheme and the Keccak oracle hash. The verification-key byte layout is flavor- and version-sensitive, and the verifier parses it as a fixed-length positional structure (1760 bytes) with no version marker or field tags: it expects the Keccak-flavor layout with the pairing-point accumulator placed after the user public inputs. Incompatible settings or toolchain versions can produce different layouts that are rejected by the verifier's fixed-length or metadata checks. Such mismatches fail closed and do not admit forged proofs.

Integrators: Contracts that build on the verifier are responsible for the semantics of a successful verification.

- **No recursive verification.** The verifier supports only non-recursive circuits. When a circuit itself verifies an inner proof, it defers the inner proof's final pairing check into a 16-element pairing-point accumulator carried in the outer proof. Matching the upstream Keccak flavor, this verifier binds the accumulator in the transcript but does not discharge the deferred pairing relation. Circuits that contain a recursive verifier must therefore not rely on this verifier to establish the validity of the accumulated inner-proof pairing; none of the in-scope circuits does so.
- **No witness confidentiality.** The supported flavor is non-ZK and provides no witness-hiding guarantee. Deployments whose security depends on witness privacy, such as a mixer note or identity preimage, must not rely on the proof for confidentiality. Proofs of the zero-knowledge flavor have a different proof structure and are rejected by the fixed proof layout.
- **No caller binding or replay protection.** A proof attests only that the circuit statement holds for the given public inputs, and neither layer ties that statement to a submitter. The wrappers perform a stateless check with no caller binding or replay state, so a valid proof may be transferable or replayable. Any consumer that treats verification success as authorization must add both halves: a circuit public input that binds the intended caller (or a nullifier), and contract logic that checks it against the authenticated invoker and enforces replay protection.
- **Proof bytes are not unique identifiers.** Multiple distinct byte encodings of the same proof verify identically because G1 padding is discarded during point decoding and non-canonical scalar encodings are normalized before transcript absorption. Consumers must therefore not use raw proof bytes, or their hash, as a security-sensitive uniqueness mechanism. Deduplication or replay protection should instead rely on an appropriate one-time public value or another explicitly defined canonical identifier.

Privileged Roles

The system has no ongoing privileged roles: no admin, owner, upgrader, or operator exists on either contract. The only privileged action is the one-shot constructor described above, and the only other entry points are the permissionless verify functions and the read-only `vk_bytes` getter.

Low Severity

L-01 Ignored Limb Padding Makes Proof Bytes Malleable for G1 Coordinates

Proof G1 points are encoded as four 32-byte limbs `(x_lo, x_hi, y_lo, y_hi)`, but the `combine_limbs` helper of `utils.rs` reconstructs each coordinate from only the last 17 bytes of the low limb and the last 15 bytes of the high limb, discarding the remaining 15 and 17 bytes without checking that they are zero. The transcript absorbs the reconstructed point rather than the raw limbs, so flipping any discarded byte leaves the parsed point, the transcript, and the verification result unchanged while producing a distinct byte string. Any consumer that treats the proof bytes, or a hash of them, as a unique identifier, for deduplication, replay protection, or signatures, can be bypassed by such a re-encoding.

This also diverges from the reference implementation the crate targets, Barretenberg `v0.87.0`, which never ignores these bytes: its limb recombination asserts `lo < 2^136` and `hi < 2^118` in assert-enabled builds, and in release builds uses every bit of both limbs, so non-zero padding alters the reconstructed point and verification fails. An exhaustive single-byte sweep confirmed 2,368 positions per proof (`37 points * 2 coordinates * (15 + 17) bytes`) that this verifier accepts and Barretenberg `v0.87.0` rejects.

Consider enforcing canonical limb encodings in `load_proof` by rejecting any G1 encoding whose low limb is not below `2^136` or whose high limb is not below `2^118` before transcript generation.

Update: *Resolved in PR 41 by commit 08f5e93.*

L-02 Non-Canonical Scalar Encodings Make Proof Bytes Malleable

Each 32-byte scalar in a proof is decoded through the `fr_word32` helper of `utils.rs` and `Fr::from_array`, which reduces the value modulo the scalar field order `r` instead of rejecting non-canonical encodings, and the Fiat-Shamir transcript absorbs the reduced value rather than the raw bytes. Every scalar slot therefore also accepts the non-canonical encodings `v + k·r`. For `v < r`, this always includes `k = 1, ..., 4`, and also includes `k = 5`

whenever `v + 5r < 2^{256}`. All such encodings verify identically, as confirmed in testing. A valid proof thus has many distinct byte encodings, so any consumer that treats the proof bytes, or a hash of them, as unique, for deduplication, replay protection, nullifiers, or signatures, can be bypassed by a re-encoding. Barretenberg `0.87.0` behaves the same way, so the port matches its reference version, although upstream has since started rejecting encodings at or above the modulus (pull request #22311 of `aztec-packages`).

Consider rejecting non-canonical encodings at the decode site by requiring each 32-byte chunk to round-trip through decoding and re-serialization, which aligns with Barretenberg's later patch. Alternatively, if byte-for-byte parity with version `0.87.0` is the priority, consider documenting that raw proof bytes are not a unique identifier and that proof identity must be derived from a canonicalized parsed representation.

Update: *Resolved in PR 41 by commit 08f5e93.*

L-03 Conflicting Barretenberg Version Claims Leave the Supported Toolchain Ambiguous

The crate's documents disagree on the Barretenberg version the verifier targets: the README names "Nargo 1.0.0-beta.9 + Barretenberg (bb v0.87.0)", while `VERIFIER_PROVENANCE.md` declares `aztec-packages` tag `v0.82.2` as the source of truth, and several module headers cite one version or the other. The development team confirmed the port was written from `v0.82.2`, yet the serialization surface matches `v0.87.0` defaults: `load_proof` expects the 456-field layout that `v0.87.0` emits by default, while `v0.82.2` emits it only when invoked as `bb prove -s ultra_honk --oracle_hash keccak --init_kzg_accumulator`. Both versions therefore work, each with its own verification key, but nothing documents the flag, and mixing a proof from one version with a verification key from the other fails at the final pairing check. The provenance document also names the implemented flavor as `UltraFlavor` when it is Barretenberg's distinct `UltraKeccakFlavor`, whose flavor-specific four-word verification-key header `load_vk_from_bytes` actually parses.

An integrator on `v0.82.2` defaults sees valid proofs rejected with no documentation pointing at the required flag, and a maintainer re-validating the port against the wrong tag can wrongly conclude it is broken. All such failures are visible: a wrong-version proof fails the proof-length check, a wrong-format verification key is rejected as `WrongLength`, and a cross-version proof/key pair fails the final pairing check; none silently verifies or misparses. The cost is confusion and wasted effort rather than a missed defect.

Consider declaring a single target toolchain across the README, the provenance document, and the module headers, with `v0.87.0` the natural choice since it matches the serialization surface without extra flags. If `v0.82.2` compatibility is also claimed, consider documenting the `--init_kzg_accumulator` invocation and the requirement that proofs and verification keys come from the same version. Consider also naming `UltraKeccakFlavor` as the implemented flavor.

Update: *Resolved in PR 41 by commit 08f5e93.*

L-04 Unvalidated BN254 G1 Points Either Trap Host Crypto or Escape Validation Entirely

The verifier deserializes G1 points from untrusted proof and verification-key bytes with no checks: the `G1Point::from_xy` and `G1Point::from_bytes` constructors wrap raw bytes in a `Bn254G1Affine`, which accepts any 64 bytes, so the only validation a point ever receives is implicit, inside the host BN254 operations.

This produces two failure modes. A malformed point with a non-zero MSM weight reaches `g1_msm` or `pairing_check`, where the host traps: the invocation fails instead of returning `VerificationFailed`, and a calling contract can contain the failure only through the `try_` invocation path, where it is indistinguishable from other host aborts. A malformed point with a zero weight, such as the padded Gemini fold commitments when the circuit is smaller than the fixed 28-round layout, is dropped by the zero-scalar filter before the host call and is therefore never validated at all. These slots are transcript-bound, so third parties cannot alter them without breaking verification, but the verifier silently accepts off-curve bytes in them.

Consider validating every G1 point in `load_proof` and returning `Result` for malformed inputs. Validating at parse time closes both failure modes at once, since the zero-scalar filter can then no longer hide an unvalidated point. This also matches the reference implementation, whose point deserialization checks curve membership at parse time.

Update: *Resolved in PR 41 by commit 08f5e93.*

L-05 Proofs Are Documented as ZK While the Verifier Implements Only the Non-ZK Flavor

The supported proof system is explicitly non-ZK: `VERIFIER_PROVENANCE.md` scopes the port to the "Non-ZK, non-recursive, native BN254 UltraHonk path only" and marks

`UltraZKFlavor` as not implemented. The supported `Proof` therefore carries witness-dependent protocol messages without a zero-knowledge guarantee, and the on-chain wrappers accept the full proof as transaction input, for example in `UltraHonkVerifierContract::verify_proof`, making those messages public in ordinary deployments.

The user-facing documentation can nevertheless lead integrators to expect privacy. The root README instructs readers to simulate the verifier using "ZK proofs", while the crate README does not state that the supported flavor is non-ZK or warn that it does not provide witness hiding. This is particularly misleading alongside privacy-oriented examples: `circuits/tornado/src/main.nr` declares `nullifier`, `secret`, and the Merkle path as private inputs, while `contracts/identity/README.md` describes `preimage` as a secret such as a password, seed, or identity secret. How much information a non-ZK proof reveals depends on the circuit, and no concrete secret recovery is demonstrated here, but the supported flavor provides no zero-knowledge guarantee protecting those values.

Consider correcting the root README so these artifacts are not described as ZK proofs, and adding prominent warnings to the crate and contract documentation that the supported flavor does not provide witness hiding. Consider also making the distinction visible at the verification interface or in API naming, and not presenting the verifier as suitable for privacy-sensitive circuits unless a ZK-capable flavor is implemented.

Update: *Resolved in PR 41 by commit 08f5e93.*

Notes & Additional Information

N-01 Misleading Comments

Several comments in the verifier crate misdescribe the values or computations they annotate. In particular:

- The `LHS_G2_BYTES` constant of `ec.rs` is described as the negated G2 generator `-[1]_2`. Negation preserves the x-coordinate, and this constant's x differs from that of the adjacent canonical generator, so it is a different point: the structured-reference-string element `[x]_2` (Barretenberg's `vk.g2_x`). Consider relabeling it as `[x]_2` and stating its derivation, so the value can be reproduced rather than taken on trust.

- The `neg_half` constant of `field.rs` is documented as `(p - 1)/2`, but `p` conventionally denotes the base-field modulus. The stored value is `(r - 1)/2`, that is, `-1/2` modulo the scalar-field modulus `r`, and is used as such. Consider documenting it as `(r - 1)/2`, naming `r` as the scalar-field modulus.
- The barycentric comment in `sumcheck.rs` writes `Σ u_i * inv_denom_i`, reusing `u`, which elsewhere denotes the round challenge, for the round-polynomial values the loop actually sums. Consider writing `Σ y_i * inv_denom_i`.
- The Shplemini comments write `r^0` and `r^j` for entries of `r_pows`, where `r_pows[j] = r^{2^j}`; read literally, `r^0 = 1` is wrong. Consider writing `r`, `r_pows[j]`, or `r^{2^j}` instead of exponents of `r`.
- The Shplemini docstring enumerates nine steps while the inline comments in the function body are numbered one to twelve, so a step traced by number can land on the wrong block. Consider keeping a single numbering scheme, or mapping each inline label to its docstring step.

None of these inaccuracies affects verification. However, a reviewer checking the arithmetic against the comments can be misled, and a future contributor could modify a correct constant to match an incorrect description. The `LHS_G2_BYTES` mislabel is the riskiest of the set, since replacing the bytes with the true `-[1]_2` would silently change the pairing equation.

Update: *Resolved in PR 41 by commit 15703c1.*

N-02 Identity Example Does Not Provide Caller Binding or Replay Protection

The identity circuit proves only that whoever generated the proof knew some `s` with `Poseidon2(s) = h`, where `h` is the public input. `prove_identity` loads the stored VK, verifies caller-supplied `public_inputs` and `proof_bytes`, and returns `Ok(())`. It reads no caller address, calls no `require_auth`, consumes no nonce, checks no expiry, and writes no state. A successful call therefore establishes only that the proof's author knew some preimage, which is weaker than the README claim that the function verifies **the caller** knows the preimage. The same README also instructs users to verify the stored VK before trusting proofs, which ensures that the intended circuit was verified but does not establish that the transaction caller is the party who knows its witness.

Because both arguments are ordinary invocation parameters, a valid proof is freely transferable. Alice can call `prove_identity(public_inputs, proof_bytes)` successfully, after which any observer can copy those values and make the same call without learning the preimage or breaking the proof system. This holds regardless of the secret's

entropy and defeats proof-of-current-possession even for a uniformly random high-entropy preimage. Impact is bounded by what the contract does with a successful verification, which is nothing: `prove_identity` reads the stored VK, verifies, and returns `Ok(())` without writing state, moving value, or granting a capability. A replayed proof therefore achieves nothing on chain today, and the exposure is to a future integration that treats a successful call as caller authentication.

Consider binding the proof to the invocation using a contract-checked public challenge containing an authenticated caller identifier and, where freshness or replay resistance is required, a contract-issued nonce consumed after successful verification. The contract should derive or validate that challenge rather than accept an arbitrary prover-supplied value. The challenge must be a public input to the circuit: a nonce the circuit never sees can prevent repeated transaction execution but does not bind the proof itself, while a nonce without caller binding still leaves the proof transferable before it is consumed. If caller binding is intentionally out of scope for this reference implementation, update the README to state that the proof establishes only knowledge of a preimage and must not be used as caller authentication.

Update: *Resolved in PR 41 by commit 08f5e93.*

N-03 The Verifier API Carries Two `Env` Handles and Mixes Them Within One Call

`UltraHonkVerifier` stores an `Env`, yet `verify` accepts a second one and alternates between the two, using the parameter for proof loading and sumcheck but the stored handle for the transcript and Shplemini. Soroban crypto types embed an `Env` by value and dispatch host calls through it, so objects from different environments can meet in one expression; `new_with_vk` opens the same gap by accepting a pre-built `VerificationKey` from anywhere.

On-chain, a contract execution interacts with exactly one host environment: the host is created per outermost invocation (CAP-46-01), inner calls share it, and a guest has no mechanism to obtain or reference objects from any other host.

Consider removing the `Env` parameter from `verify` and using the stored handle throughout, making the mismatch unrepresentable, and either making `new_with_vk` internal or documenting that its `VerificationKey` must come from the same `Env`.

Update: *Resolved in PR 41 by commit 08f5e93.*

N-04 `lookup_read_tags` Is Not Constrained to Be Boolean

The log-derivative lookup relation in `accumulate_log_derivative_lookup_relation` builds `inv_exists = lookup_read_tags + q_lookup - lookup_read_tags * q_lookup`, which is the logical OR of the two operands only when both are boolean. `q_lookup` is a verification-key selector, but `lookup_read_tags` is a prover-supplied witness, and no subrelation constrains it to `{0, 1}`, and it appears nowhere else in the relation set.

This is not a soundness break for this relation. Where `q_lookup = 1` the tag cancels, so an actual lookup is unaffected. Where `q_lookup = 0` a non-boolean tag only rescales that row's inverse, which is absorbed by rescaling `lookup_read_counts`; `read_term` and `write_term` are untouched. Since `write_term` is built from the fixed verification-key table polynomials, this cannot create a denominator for a value outside the table, so membership and multiplicity still follow from the log-derivative sum (Schwartz-Zippel). The freedom is a redundant re-parameterization, not an exploit, as Barretenberg's own post-fix comment notes: "tampering can only skip a `table_term`, which is disadvantageous for the cheating prover."

The implementation matches Barretenberg `v0.87.0`, which omitted the constraint. Barretenberg later added the booleanity subrelation `lookup_read_tags * lookup_read_tags - lookup_read_tags = 0` in PR #15007 (merged 2025-06-18, resolving issue #1405). Adding it here would change the batched relation set and so needs matching prover support. While `v0.87.0` remains the reference, consider documenting that `lookup_read_tags` is intended to be boolean but is not independently constrained, and that it is not the same as `q_lookup` (a legitimate row may have `q_lookup = 0` and `lookup_read_tags = 1`). If the prover and verifier are upgraded together, move to a Barretenberg version that includes the check.

Update: *Resolved in PR 41 by commit 08f5e93.*

N-05 Unused Proof Padding Affects Fiat-Shamir Challenges Without Being Constrained

The proof layout is fixed at `CONST_PROOF_SIZE_LOG_N = 28` slots regardless of `vk.log_circuit_size`, and the transcript absorbs every slot: all 28 sumcheck rounds in `generate_sumcheck_challenges`, all 27 fold commitments in `generate_gemini_r_challenge`, and all 28 Gemini evaluations in `generate_shplonk_nu_challenge`. Verification, however, constrains only the `log_n`

prefixes: `verify_sumcheck` loops over `0..log_n`, and the unused fold commitments keep zero MSM scalars. For `log_n < 28`, the trailing entries are therefore hashed but never algebraically checked. Barretenberg 0.87.0 uses the same fixed-size transcript construction and likewise masks padded sumcheck rounds and unused fold commitments, but its Shplemini reduction still binds padded Gemini evaluations through the constant-term accumulator, whereas this verifier ignores evaluations beyond `log_n`.

A prover can vary these algebraically unconstrained entries and regenerate the dependent proof suffix to resample the downstream challenges (`rho`, `gemini_r`, `shplonk_nu`, and `shplonk_z`). This grants additional random-oracle queries, but each attempt still faces the 128-bit challenge space, so no practical soundness break results. The honest prover emits deterministic padding: zero for unused sumcheck univariates and Gemini evaluations, and the G1 generator for unused fold commitments.

Consider documenting which compatibility contract the crate intends: accepting every proof Barretenberg 0.87.0 accepts, in which case arbitrary padding is part of the accepted language, or accepting only the canonical proofs its prover emits. If the latter is intended, consider rejecting non-canonical padding during parsing, after confirming the padding convention across every supported `log_circuit_size`, while preserving the fixed-slot transcript construction. That same decision also settles the non-canonical scalar-encoding case.

Update: *Resolved in PR 41 by commit 08f5e93.*

N-06 Circuit Build Inputs Are Not Immutably Pinned, So Artifact Reproducibility Is Not Guaranteed

The circuit build depends on inputs that are not immutably pinned, so unchanged repository source can compile to different ACIR and VK across rebuilds. An already-deployed verifier stays bound to its existing VK, so nothing changes on chain; the concern is reproducibility and provenance, not runtime exploitation. Three inputs are unpinned:

- **Poseidon2 is pulled through a mutable Git tag.** `identity` and `tornado` depend on `poseidon` via `tag = "v0.2.0"`. The upstream `noir-lang/poseidon` maintainer can move that tag to a different commit, and no lockfile records the resolved commit, so a rebuild can fetch different library source. Consider vendoring Poseidon2 into the repository and depending on it by `path`: nargo has no commit-`rev` key and writes no lockfile, so vendoring is the only way to freeze the exact source in-manifest. If the git

dependency is kept, a CI step could assert that `git ls-remote … v0.2.0` resolves to an expected commit before building.

- **The Noir and Barretenberg versions are declared but not enforced.** `build_all.sh` names Noir `1.0.0-beta.9` and Barretenberg `v0.87.0`, but skips installation when a binary already exists and then uses whatever is on `PATH` without a version check, so any locally installed toolchain is used. `compiler_version` cannot backstop this: nargo forbids prerelease requirements and compares its own version with the prerelease stripped, so it gates only at release granularity and cannot pin `1.0.0-beta.9`. The backend exposure is narrow: Noir betas 7 through 12 each pin Barretenberg `0.87.0` in their `install_bb.sh`, so any compiler in that range targets the same backend this verifier ports, and beta.9 is the version it is mainly tested against. The residual risk is a compiler outside that range, or a different Barretenberg, silently producing artifacts that diverge from the reviewed ones. Consider installing the tested toolchain (Noir `beta.9`, Barretenberg `0.87.0`) in the build pipeline instead of skipping when a binary is present, and asserting `nargo --version` and `bb --version` before compiling — at minimum within the `beta.7`–`beta.12` range known to target Barretenberg `0.87.0`.

- **Rebuilt artifacts are never checked against the reviewed ones.** Nothing compares regenerated circuits and VKs to known-good values, so drift in either input above passes silently. Consider having CI regenerate each circuit and its VK and compare their hashes against committed known-good values, failing the build on mismatch.

Update: *Resolved in PR 41 by commit 08f5e93.*

Conclusion

The audited codebase provides an on-chain verifier for UltraHonk proofs of Noir circuits on Stellar's Soroban platform: a Rust `no_std` port of the Barretenberg v0.87.0 verifier in its non-zero-knowledge, Keccak-transcript flavor, together with two thin contract wrappers holding immutable verification keys and eight example circuits. OpenZeppelin audited the verifier crate, both wrappers, and all eight circuits at a single pinned commit.

The review centered on a differential comparison against the Barretenberg reference. The transcript and challenge derivation, the full Ultra relation set, the Shplemini batch opening, and the final pairing check were found to follow the upstream algorithm, and no exploitable verifier-specific soundness divergence was identified in the core verification equations. Delegating the field arithmetic, multi-scalar multiplication, pairing, and hashing to the Soroban host keeps the hand-written cryptographic surface small. The in-scope circuits were reviewed for under-constrained witnesses, and no circuit-level soundness issue was identified.

The issues that were identified sit primarily at the edges of the design rather than in the core verification equations. A proof admits multiple byte encodings that verify identically, since padding bytes dropped during point decoding and non-canonical scalar encodings are not bound by the transcript, so consumers must not key deduplication or replay protection on raw proof bytes. More broadly, for supported non-recursive circuits, a successful verification establishes that the circuit relation holds for the given public inputs; it does not bind the proof to the submitter, provide replay protection, or provide witness confidentiality in the supported non-ZK flavor. These properties place meaningful obligations on integrating contracts, and documenting them prominently at the contract level would reduce the risk of downstream deployments assuming guarantees the verifier does not provide.

We thank the Nethermind team and the Stellar Development Foundation for facilitating the engagement and for their support throughout the review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.